UNITED INTERNATIONAL UNIVERSITY

Department of Computer Science & Engineering

School of Science & Engineering

PROJECT MANAGEMENT REPORT

UIU CampusConnect

Campus Social & Academic Networking Platform

Vision & Scope • Development Methodology
Risk Management & RMMM • Work Breakdown Structure

Team No. 09 | Section: B

S/N	Name of Team Member	Student ID
1	Sindeed Shuaib Arpon	0112330261
2	Nafiz Iqbal Razin	0112330204
3	Afnan Zaman Mohammad Sajid	0112330036
4	Farzana Akter	0112320101

Date of Submission: August 23, 2026

Contents

1 Introduction **1**
- 1.1 Project Overview . . . 1
- 1.2 Purpose and Structure of this Report . . . 1

2 Vision and Scope **2**
- 2.1 Vision Statement . . . 2
- 2.2 Project Objectives . . . 2
- 2.3 In-Scope Functional Areas . . . 3
- 2.4 Out-of-Scope Items . . . 5
- 2.5 Assumptions and Constraints . . . 6

3 Development Methodology **7**
- 3.1 Selecting a Process Model for CampusConnect . . . 7
- 3.2 Comparison of Candidate Methodologies . . . 7
- 3.3 Selected Methodology: Agile Scrum . . . 8
  - 3.3.1 Scrum Roles in Team 09 . . . 8
  - 3.3.2 Applying the Scrum Cycle to UIU CampusConnect . . . 9
  - 3.3.3 Indicative Sprint Roadmap . . . 9
- 3.4 Why the Other Methodologies Were Not Chosen . . . 9
  - 3.4.1 Why not Waterfall . . . 9
  - 3.4.2 Why not the Prototype model . . . 11
  - 3.4.3 Why not the Spiral model . . . 11
  - 3.4.4 Why not Agile without Scrum . . . 11

4 Risk Management and RMMM **13**
- 4.1 The Concept of Risk in this Project . . . 13
- 4.2 Risk Categories in UIU CampusConnect . . . 13
- 4.3 The RMMM Strategy for UIU CampusConnect . . . 14
  - 4.3.1 Risk Mitigation: acting before the risk occurs . . . 14
  - 4.3.2 Risk Monitoring: watching while the project runs . . . 14
  - 4.3.3 Risk Management: responding when the risk occurs . . . 14
- 4.4 Worked RMMM Examples . . . 15
- 4.5 Risk Plan Sheet . . . 16
- 4.6 Interpreting the Risk Profile . . . 17

5 Work Breakdown Structure **18**
- 5.1 Introduction . . . 18
- 5.2 Principles Applied . . . 18
- 5.3 Rules Followed . . . 19
- 5.4 Level Structure . . . 19
- 5.5 Level 1 and Level 2 Overview . . . 19
- 5.6 Four-Level Decomposition . . . 20
  - 5.6.1 1.0 Platform Core & Foundation (15%) . . . 20
  - 5.6.2 2.0 Identity & Social Network (22%) . . . 21
  - 5.6.3 3.0 Campus Feed & Engagement (20%) . . . 21
  - 5.6.4 4.0 Academic & Campus Collaboration (23%) . . . 21
  - 5.6.5 5.0 Campus Utilities & Governance (20%) . . . 22
- 5.7 Effort Roll-Up Verification . . . 22

6 Wideband Delphi Estimation **32**
6.1 What is Wideband Delphi Estimation? . . . 32
6.2 Why Wideband Delphi Suits UIU CampusConnect . . . 32
6.3 Estimation Session: Three Rounds . . . 32
6.3.1 Round 1 — Independent Estimates . . . 33
6.3.2 Round 2 — After the First Assumption Review . . . 33
6.3.3 Round 3 — Convergence . . . 33
6.4 Convergence Analysis . . . 34
6.5 Feature-Level Convergence: 4.2 Campus Events & Attendance . . . 35
6.5.1 What Happened in Each Round . . . 35
6.6 Assembling the Tasks . . . 36
6.7 Reviewing the Results . . . 36
6.8 Limitations of the Estimate . . . 38

7 Conclusion **39**

CHAPTER 1

Introduction

1.1 Project Overview

UIU CampusConnect is a university-focused digital platform that brings social networking, academic communication, campus notices, events, clubs, peer-to-peer services and administrative functions into a single shared environment. At present a student at United International University has to move between a notice board, a departmental portal, several unofficial Facebook groups, a handful of WhatsApp or Messenger threads and word-of-mouth in order to keep track of what is happening on campus. Information is duplicated, arrives late, and is frequently impossible to verify. CampusConnect exists to remove that fragmentation.

The platform is built around role-based authentication, personalised feeds, profiles, private messaging, notifications, official notices, academic section announcements, events with registration and QR check-in, club management, course and project groups, a campus marketplace, lost-and-found, global search, moderation and system administration. Every one of these modules is treated as a managed deliverable in this report: it is scoped in Chapter 2, scheduled through the methodology described in Chapter 3, protected by the risk plan in Chapter 4, and decomposed into work packages in Chapter 5.

1.2 Purpose and Structure of this Report

This document is the consolidated project management report for UIU CampusConnect. It is not a design specification and it is not a user manual; it records the management decisions that govern how the system will be built, by whom, in what order, at what cost, and with what protection against failure. The report is organised as follows.

- **Chapter 2 – Vision and Scope** fixes the boundary of the product. It states what CampusConnect is for, lists the objectives it must achieve, defines the functional areas that are inside the delivery boundary, and explicitly records what the team will *not* build.
- **Chapter 3 – Development Methodology** compares the candidate process models against the actual characteristics of this project, justifies the selection of Agile Scrum, gives a reasoned explanation of why each alternative was set aside, and shows how Scrum ceremonies map onto the CampusConnect modules.
- **Chapter 4 – Risk Management and RMMM** identifies the risks that threaten the platform, defines the team's Risk Mitigation, Monitoring and Management procedure, works three representative risks through that procedure in full, and consolidates all identified risks into a risk plan sheet.
- **Chapter 5 – Work Breakdown Structure** decomposes the entire scope into four hierarchical levels, from the platform itself down to individually assignable work packages, with effort allocations that roll up to 100% of the project.
- **Chapter 6 – Wideband Delphi Estimation** converts that structure into absolute effort. Five estimators worked through three rounds of consensus estimation; the chapter records every round, analyses how the estimates converged, and works one feature's estimation session through in full.

CHAPTER 2

Vision and Scope

2.1 Vision Statement

> To create a unified digital campus ecosystem that connects students, faculty, university officials and campus organisations through social interaction, academic collaboration, communication and community services.

Each clause of this statement carries a concrete obligation for the CampusConnect build. *Unified* means that a student signs in once and reaches notices, sections, clubs, events and peers from a single feed, rather than maintaining accounts on several disconnected services. *Digital campus ecosystem* means the platform mirrors the real structure of the university – departments, sections, batches, clubs and offices are first-class entities in the data model, not free-text labels. *Connects students, faculty, university officials and campus organisations* is the reason the system is role-based from the very first sprint: a notice published by the Registrar's office must be visibly different from a post written by a classmate, and the platform must be able to prove which is which. Finally, *social interaction, academic collaboration, communication and community services* names the four pillars that later become the top-level branches of the Work Breakdown Structure in Chapter 5.

2.2 Project Objectives

The vision is delivered through six measurable objectives. Each objective below is stated first, then explained in terms of the specific CampusConnect features that satisfy it.

O1. Improve the speed and reach of university communication

Official information at UIU currently travels through physical notice boards, email and informal re-sharing, which means a student who misses one channel misses the announcement entirely. CampusConnect replaces that with the *Official University Notice Board*, where an authorised office publishes once and the notice is targeted to the exact audience it concerns – a department, a batch, a section or the whole campus – and is pushed instantly through the notification centre. Because notices are stored, tagged by priority and searchable, reach is no longer dependent on a student being online at the moment of publication. This objective is realised by WBS elements 4.1.1 and 2.2.2.

O2. Provide a campus-specific social and networking environment

General-purpose social networks mix campus life with unrelated content and admit anyone. CampusConnect restricts registration to verified university email addresses, so every account belongs to a real member of the UIU community. On top of that trusted membership the platform provides the campus feed, posts with rich media, reactions, comments, polls and saved content – the same social affordances students already expect, but scoped to people they actually share a campus with. This objective is realised by WBS elements 2.1.1, 3.1 and 3.2.

O3. Support academic interaction between students and faculty

Academic contact today depends on catching a teacher in the corridor or hoping a reply arrives by email. CampusConnect gives faculty structured profiles carrying office hours, course allocation and research interests, and gives every course section a channel through which the assigned teacher can post announcements that reach exactly the enrolled students. Course, batch and project groups extend the same idea to peer learning, with a shared file area for slides, notes and project documents. This objective is realised by WBS elements 2.1.2, 4.1.2 and 4.3.2.

O4. Centralise events, clubs, notices and community activities

Club recruitment, seminars, workshops and cultural events are currently advertised in whichever group the organiser happens to have access to, and attendance is counted on paper. CampusConnect provides a single events module with publishing, capacity limits, interested/going responses, registration and QR-token check-in, together with a club directory in which every registered club maintains its own profile, membership list, content and gallery. The result is one authoritative campus calendar and one verifiable attendance record. This objective is realised by WBS elements 4.2 and 4.3.1.

O5. Provide peer-to-peer services such as marketplace and lost-and-found

Students routinely resell textbooks, calculators and lab equipment, and lose belongings on campus, but have no trusted place to do either. CampusConnect adds a campus marketplace where verified students list items with photographs, category and price and mark them sold when the exchange is complete, and a lost-and-found register where a finder posts an item and a claimant must supply identifying details before the claim is released. Because both services run on top of verified identities, the platform can attribute every listing and claim to a real student, which is precisely what makes them safer than an anonymous group. This objective is realised by WBS element 5.1.

O6. Maintain role-based security, moderation and administrative control

A platform that carries official notices, private messages and personal profiles cannot be governed informally. CampusConnect enforces a permission matrix in which student, faculty, official and administrator roles each see and do only what their role allows; provides a reporting mechanism and a moderation centre where flagged content is reviewed and actioned; and gives administrators a dashboard with immutable audit logs of every privileged action. This objective is what makes the other five safe to deploy, and it is realised by WBS elements 2.1.1, 5.2 and 1.3.1.

2.3 In-Scope Functional Areas

Table 2.1 defines the delivery boundary of the project. The first two columns state the functional area and the functionality it contains; the third column explains why that area belongs in CampusConnect, so that the scope can be defended when new requests arrive during development.

Table 2.1: In-scope functional areas of UIU CampusConnect and their justification

Area	Included Functionality	Role in the CampusConnect Project
User management	Registration, authentication, role-based permissions, profiles and privacy controls.	The trust anchor of the whole platform. Restricting sign-up to university email domains is what separates CampusConnect from an open social network, and the role assigned at registration decides whether a user may publish an official notice, moderate content or only participate. Nothing else can be built safely until this is in place, so it is the first module scheduled.

continued on next page

Area	Included Functionality	Role in the CampusConnect Project
Social networking	Campus feed, personalised views, posts, media, reactions, comments, polls and saved content.	The layer that gives students a reason to open the application daily. A personalised feed mixing department, batch and followed sources keeps the platform relevant to each individual, while polls and saved content turn passive reading into participation and let clubs and class representatives gather opinion quickly.
Communication	Connections, following, private messaging and notifications.	Replaces the scattered Messenger and WhatsApp threads students currently use for coursework coordination. Because messaging sits inside a verified campus identity system, a student can be certain who they are talking to, and the central notification service ensures that a notice, an event reminder and a message all reach the user through one consistent channel.
Academic support	Faculty profiles, office hours, research/course information, academic resources and section announcements.	Turns the platform from a social product into an academic one. Section-level announcements guarantee that a change of class time or an assignment deadline reaches every enrolled student and no one else, and published office hours reduce the repeated individual queries that faculty currently receive.
Official communication	Authorised university notices with targeting, priority and filtering.	Gives the university itself a stake in the system. Only accounts with the official role may publish here, which makes the notice board authoritative and prevents the rumour-driven misinformation that circulates in unofficial student groups before examinations and registration.
Events	Event publishing, registration, interest/going actions, capacity and QR/token-based check-in.	Solves a real operational problem for club organisers and the Office of Student Affairs: knowing in advance how many people will attend, enforcing venue capacity, and recording attendance without paper sign-in sheets. QR check-in also produces the participation data clubs need for their annual reports.

continued on next page

Area	Included Functionality	Role in the CampusConnect Project
Clubs	Club directory, membership requests, club content, events and galleries.	UIU's clubs are a major part of campus life but have no permanent online presence. Giving each club a managed profile with a membership workflow means recruitment, announcements and archives survive the annual change of committee instead of disappearing with a graduating panel.
Groups & resources	Course, batch, project and community groups with shared files.	Supports the collaborative work that dominates a CSE student's semester. A project group with a shared file area and a discussion board keeps code documents, reports and decisions in one place, which is where most team coursework currently breaks down.
Community services	Marketplace and lost-and-found functionality.	The peer-to-peer services of objective O5. They are deliberately in scope because they generate everyday, non-academic reasons to return to the platform, which is what sustains adoption after the novelty of launch.
Search & moderation	Global search, reporting, moderation centre and administrator controls.	The governance layer. Global search makes the accumulated notices, events and listings retrievable rather than merely chronological, while reporting and moderation give the team a defined response path when a user posts abusive content or a fraudulent marketplace listing.

2.4 Out-of-Scope Items

The following capabilities are explicitly excluded from this release. Recording them here is a scope-control measure: it gives the project manager a written basis for refusing late additions, which is the main defence against the scope creep risk registered as R9 in Chapter 4.

- **Full Learning Management System replacement.** CampusConnect carries section announcements and shared group files, but it will not host graded assignment submission, grade books or course content delivery. UIU already operates an LMS for that purpose, and reproducing it would multiply the project's size while duplicating a system the university has no intention of retiring.
- **Online examination and proctoring system.** Examinations demand identity assurance, secure browser control, invigilation and legally defensible audit trails. These requirements are of a different order from social and communication features, and attempting them within a student project timeline would put the delivery of the entire platform at risk.
- **Video-conferencing platform.** Real-time audio and video demands media servers, bandwidth provisioning and continuous operational cost. CampusConnect therefore supports scheduling and announcing an online session and linking out to an existing conferencing tool, rather than transporting the media itself.

- **University-wide ERP or finance system replacement.** Admissions, payroll, accounts and academic records remain in the university's existing administrative systems. CampusConnect may consume identity and section data from them, but it does not become the system of record for institutional data.
- **Direct financial transaction processing and payment gateway services.** The marketplace connects a buyer and a seller and lets them mark an item sold; money changes hands offline. Handling payments would bring PCI-DSS-level compliance obligations, chargeback handling and custody of funds into a student project, which is neither necessary for the objective nor responsibly achievable within it.

2.5 Assumptions and Constraints

The scope above is valid only while the following assumptions hold, and it is bounded by the constraints listed with them.

- **Assumption – institutional email verification is available.** Role-based access depends on being able to verify a `@uiu.ac.bd` or `@bscse.uiu.ac.bd` address at registration. If that check is unavailable, the fake-account risk (R5) rises sharply and a manual approval step must be added.
- **Assumption – section and club data can be obtained.** Section announcements and club directories assume the team can import, or manually seed, an accurate list of sections, courses and registered clubs.
- **Constraint – academic calendar.** The project is executed inside a single semester by a four-member team, which caps total available effort and is the reason the WBS enforces small, individually ownable work packages.
- **Constraint – no operational budget.** Deployment targets free or low-cost hosting tiers, which directly shapes the horizontal-scaling mitigation chosen for the peak-load risk (R3).

CHAPTER 3

Development Methodology

3.1 Selecting a Process Model for CampusConnect

A software methodology is the organised way in which a team moves a product from an idea to a working, maintained system. The choice is not academic: it fixes how requirements are captured, how often working software appears, how change is absorbed and how risk is discovered. For CampusConnect three project characteristics dominate the decision.

1. **The requirements will move.** The platform serves students, faculty, club organisers and administrative offices, and none of these groups can fully specify what they want before they see it. A model that freezes requirements before construction would guarantee that the delivered system is the wrong one.
2. **The system is modular but interdependent.** Ten functional areas can largely be built in parallel, yet they all sit on the same identity, notification and moderation foundations. The process must therefore allow parallel work while forcing frequent integration.
3. **Effort is fixed and small.** Four students working across one semester cannot absorb a heavy documentation overhead, and cannot recover from discovering a fundamental defect late.

3.2 Comparison of Candidate Methodologies

Table 3.1 evaluates the four candidate models against those three characteristics.

Table 3.1: Candidate process models assessed against the needs of UIU CampusConnect

Model	Core Idea	Principal Limitation	Fit for CampusConnect
Waterfall	Strictly sequential phases – requirements, design, implementation, testing, deployment, maintenance – with no return to a completed phase. Produces heavy, complete documentation.	Rigid. Change after sign-off is expensive, and testing happens so late that major defects surface when there is no time to absorb them.	**Rejected.** It would require students, faculty and the administration to agree a complete specification for ten modules before any code exists, and would leave the first working demonstration until the end of the semester.
Prototype	Build a simplified but functional model of the system, show it to users, collect feedback and refine until the requirements are understood.	It is a requirements-elicitation technique, not a complete construction process; it has no built-in answer for scheduling, integration or release.	**Partially adopted.** Prototyping is genuinely useful for the feed and event check-in interfaces, so it is used *inside* sprints as a design activity, not as the overall project model.

continued on next page

Model	Core Idea	Principal Limitation	Fit for CampusConnect
Spiral	Repeated cycles of planning, risk analysis, engineering and evaluation, with an explicit risk-driven decision at every turn.	Costly and heavy to administer; the ceremony only pays for itself when the consequences of failure are severe.	**Rejected as the overall model.** CampusConnect is risk-aware but not safety-critical or contractually exposed. Its risk discipline is instead supplied by the RMMM process of Chapter 4, executed inside sprints.
Agile Scrum	Iterative and incremental delivery in short sprints from a prioritised product backlog, with defined roles and ceremonies and working software as the measure of progress.	Requires committed, reasonably skilled participation from everyone, and can invite scope creep if the backlog is not controlled.	**Selected.** It matches all three project characteristics: it expects changing requirements, forces integration every sprint, and produces something demonstrable long before the deadline.

3.3 Selected Methodology: Agile Scrum

Agile Scrum is adopted as the development methodology for UIU CampusConnect. The decision rests on four properties of Scrum, each of which answers a specific problem in this project.

Short, timeboxed sprints. Work is delivered in two-week sprints. Within one semester this yields roughly six sprints, so the team reaches a demonstrable, testable increment of CampusConnect within the first month rather than at the end. A defect in the authentication or notification foundation is therefore discovered while there is still time to correct it.

A prioritised product backlog. The ten functional areas are not equally urgent. The backlog lets the team build authentication, profiles and the feed first because everything else depends on them, and defer the marketplace and lost-and-found until the core is stable – without ever pretending that the deferred items have been forgotten.

Frequent stakeholder feedback. Each sprint ends with a demonstration to the supervising faculty member and to sample users drawn from the student body and club committees. Questions such as whether a notice should be filtered by department or by section are answered by people who will actually use the system, in the sprint after they are raised.

Clear ownership through defined roles. With only four members, ambiguity about who owns a module is fatal. Scrum's role structure, combined with the single-point accountability rule applied in the WBS, ensures every work package has exactly one owner.

3.3.1 Scrum Roles in Team 09

Scrum defines roles rather than a management hierarchy, and in a four-member student team the distinction matters. Nobody is released from development work to hold a role: every role holder still writes code. What the roles fix is *decision rights* — who orders the backlog, who removes impediments, and who is accountable for the increment — so that a decision never stalls waiting for the whole group to agree. Table 3.2 records how the three roles are allocated and what each one actually does on this project.

Table 3.2: Allocation of Scrum roles within the CampusConnect team

Role	Responsibility	Application in CampusConnect
Product Owner	Owns and orders the backlog; represents the users.	Collects requirements from students, club representatives and the supervising faculty member, and decides sprint-by-sprint which of the ten modules is next.
Scrum Master	Facilitates ceremonies and removes impediments.	Runs the daily stand-up and the retrospective, and maintains the risk register described in Chapter 4, escalating any risk whose exposure rises.
Development Team	Designs, builds, integrates and tests the increment.	All four members; each takes ownership of specific WBS work packages and is accountable for their integration and testing.

3.3.2 Applying the Scrum Cycle to UIU CampusConnect

The seven activities in Table 3.3 are what the team repeats every two weeks. Each is described in terms of the CampusConnect artefacts it consumes and produces, because a ceremony that is not tied to a real artefact quickly becomes a ritual performed for its own sake. Two of them carry most of the weight for this project: the sprint review, which is the only point at which real students see the software and correct our assumptions about it, and the retrospective, which is where realised risks are examined and mitigations updated.

3.3.3 Indicative Sprint Roadmap

The roadmap in Table 3.4 is indicative rather than contractual — in Scrum the backlog is re-ordered at every sprint planning session — but it records the dependency order the team intends to follow, and it is the basis for the capacity check performed in Chapter 6. The ordering is driven by dependency rather than by preference: nothing can be built until identity and roles exist, nothing can be moderated until posts exist, and the load and security work is deliberately placed last because it can only test a system that is already complete.

3.4 Why the Other Methodologies Were Not Chosen

Selecting Agile Scrum is only half of the decision; the other half is being able to state why each alternative was set aside. The four subsections below give that reasoning in terms of UIU CampusConnect rather than in the abstract.

3.4.1 Why not Waterfall

Waterfall would require the team to freeze a complete requirements specification for all ten functional areas before writing any code, and to accept that no phase could be revisited once signed off. Three properties of this project make that untenable.

First, the requirements genuinely are not knowable in advance. Whether a section announcement should reach the whole department or only the enrolled students, whether marketplace listings should expire automatically, and how aggressively the feed should rank departmental content are all questions that our sprint reviewers answered only after using a working screen. Under Waterfall those answers would arrive after the design phase had closed, and changing them would mean formal rework of documents that were already approved.

Second, Waterfall places testing at the end. For a platform whose highest-exposure risks are a privacy

Table 3.3: Scrum activities mapped onto the CampusConnect development cycle

Activity	Application to UIU CampusConnect
Product Backlog	Collect and prioritise platform requirements – authentication, profiles, feed, messaging, notifications, notices, events, clubs, groups, marketplace, lost-and-found, search and moderation – expressed as user stories with acceptance criteria and traced to WBS codes.
Sprint Planning	Select a realistic group of high-priority backlog items for the next two-week sprint, sized against the 8–40 hour work packages defined in the WBS, and agree the sprint goal.
Development Sprint	Design, code, integrate and test the selected features. Every merge runs the automated test suite from WBS 1.2.2 so that integration defects surface within the sprint rather than at the end of the project.
Daily Scrum	A fifteen-minute stand-up covering progress, plan and impediments. New risks raised here are added to the risk register the same day.
Sprint Review	Demonstrate completed functionality to the supervising faculty member and a sample of student users, and record their feedback as new or re-prioritised backlog items.
Sprint Retrospective	Identify what worked, what did not, and one concrete process improvement for the next sprint. Realised risks are reviewed here and their mitigation strategies updated.
Release / Increment	Deploy the stable, tested increment to the staging environment, update the backlog with new learning, and re-baseline the remaining effort against the WBS.

Table 3.4: Indicative sprint roadmap for UIU CampusConnect

Sprint	Primary WBS Coverage	Sprint Goal
1	1.1, 1.2.1	Architecture, data model, UI design system and a working build pipeline.
2	2.1	Verified registration, login, role permissions and student/faculty profiles.
3	3.1, 3.2.1	Campus feed with post creation, media and visibility scoping.
4	2.2, 3.2.2, 3.2.3	Messaging, notification centre, reactions, polls and threaded comments.
5	4.1, 4.2	Official notice board, section announcements, events and QR check-in.
6	4.3, 5.1	Clubs, academic groups, marketplace and lost-and-found.
7	5.2, 1.2.2, 1.3	Moderation, admin dashboard, load testing, security audit and deployment.

breach, a peak-load failure and silent module integration defects (risks R1, R3 and R8 in Chapter 4), discovering any of them in a final test phase would leave no time in a single semester to correct them.

Third, Waterfall's strength – heavy, complete documentation and a fixed contractual baseline – buys us nothing here. There is no external client, no fixed-price contract and no regulatory approval gate that would justify the documentation overhead for a four-member student team.

Waterfall would be the correct choice for a project whose requirements are genuinely stable and externally fixed. CampusConnect is the opposite of that.

3.4.2 Why not the Prototype model

The Prototype model is attractive at first glance because CampusConnect is a heavily user-facing product and we do intend to put screens in front of students early. The problem is that prototyping is a *requirements elicitation technique*, not a complete development process. It tells the team how to discover what users want; it says nothing about how to schedule twelve components across a semester, how to integrate the work of four people, when to test, or when to release.

Used as the overall model it also carries two specific dangers for this project. Because a prototype is deliberately built to be thrown away or reworked, teams under deadline pressure frequently promote a prototype into production; for a system that stores personal data and carries official notices, shipping prototype-quality authentication would be unacceptable. And because the model invites indefinite refinement cycles driven by user feedback, it is a direct accelerant for the scope creep risk (R9) that a twelve-module platform already faces.

Our resolution is therefore not to reject prototyping but to demote it: it is used *inside* sprints as a design activity for the feed, the event check-in flow and the notice board, while Scrum supplies the surrounding process discipline.

3.4.3 Why not the Spiral model

The Spiral model is the most defensible of the alternatives, because CampusConnect genuinely is risk-sensitive and Spiral is explicitly risk-driven. It was set aside for reasons of proportionality rather than principle.

Spiral requires a formal risk analysis phase, with prototyping and evaluation, at every cycle of the project. That ceremony is justified when failure is catastrophic and irreversible – avionics, medical devices, large fixed-price government contracts – because there the cost of the analysis is trivially small against the cost of being wrong. CampusConnect is a campus platform built by four students: a defect is recoverable, a release can be rolled back, and there is no contractual exposure. Spending a meaningful share of a 126 person-day budget (Chapter 6) on formal risk cycles would consume effort that the project needs for construction.

Spiral also assumes an experienced risk analyst, which a student team does not have, and it is widely regarded as difficult to administer even in industry.

Crucially, rejecting Spiral does not mean rejecting risk discipline. We keep the part of Spiral that pays for itself – systematic, repeated risk analysis – and run it as the RMMM process of Chapter 4 inside the Scrum cycle: risks are re-scored at every sprint planning, trigger indicators are watched during the sprint, and mitigations are funded as WBS work packages. We get Spiral's risk awareness at Scrum's overhead.

3.4.4 Why not Agile without Scrum

Agile is a family of values, not a process. An unstructured Agile approach – iterating informally, responding to feedback, keeping documentation light – would suit this project's spirit but would leave three questions unanswered that Scrum answers explicitly.

- **Who decides what is built next?** Twelve components compete for a fixed budget. Scrum's Product Owner and ordered product backlog give a single, visible answer; informal Agile gives a debate.
- **When is work integrated and demonstrated?** Scrum's timeboxed sprint and sprint review force integration and stakeholder feedback on a fixed cadence. Without that cadence, four people building in

parallel would discover integration defects late – exactly risk R8.

- **What stops the backlog from growing without limit?** The sprint boundary and the formal approval step for new requirements are the team's primary control on scope creep (risk R9).

Extreme Programming was considered briefly for the same reason and set aside on practical grounds: its core practices – full-time pair programming, test-first development on every unit, and a continuously available on-site customer – assume availability that student timetables and a volunteer user base cannot supply. We adopt its useful, low-cost habits (continuous integration, automated regression tests, collective code ownership) inside Scrum rather than adopting XP wholesale.

CHAPTER 4

Risk Management and RMMM

4.1 The Concept of Risk in this Project

A risk is not a problem yet; it is a possibility of a future problem. Risk analysis and management are the actions a software team takes to understand and control that uncertainty before it becomes damage. For CampusConnect the distinction matters practically: the platform stores personal data for the entire student body, carries official university communication and will face its heaviest load precisely on the days that matter most – registration, section selection and result publication. Discovering these problems in production would be far more expensive than planning for them now.

Every risk in this project is characterised by two quantities, each scored from 0 to 5:

- **Probability** – how likely the risk is to occur during the project or in early operation.
- **Impact** – the severity of the consequence for users, data or the schedule if it does occur.

Their product is the **risk exposure**, and it is the number the team uses to decide where to spend limited effort. Exposure of 20 or above is treated as critical and must have an active mitigation in the current sprint; 10–19 is significant and is scheduled; below 10 is monitored.

4.2 Risk Categories in UIU CampusConnect

Classifying a risk is not bookkeeping; it decides who is accountable for it and where the mitigation belongs. A technical risk is answered by an engineering work package in the WBS, a project risk by a change to the plan or the backlog, a business risk by how the platform is introduced to its users, and an external risk mainly by contingency because the team cannot reduce its probability. Table 4.1 gives the four categories with the concrete forms each takes in this project.

Table 4.1: Risk categories and concrete CampusConnect examples

Category	Definition	Examples in CampusConnect
Technical risk	Arising from technology, tools and system design.	The notification service fails to deliver under load; a module's API contract changes and breaks its consumers; the server goes down when the whole batch opens the results notice at once.
Project risk	Arising from planning, scheduling and resource allocation.	A four-member team loses a member to illness mid-sprint; the semester deadline leaves no slack; the backlog keeps growing as reviewers request new features.
Business risk	Threatening the value and adoption of the product.	Students continue to use existing Facebook groups and never adopt the platform; the university declines to endorse the official notice board, removing its authority.
External risk	Originating outside the team's control.	A change in data-protection expectations forces rework of how personal data is stored; hosting provider outage; campus closure disrupting user testing.

4.3 The RMMM Strategy for UIU CampusConnect

RMMM – Risk Mitigation, Monitoring and Management – is the structured plan through which the team handles risk across the project lifecycle. Rather than restating the theory, this section describes the procedure the CampusConnect team actually follows.

4.3.1 Risk Mitigation: acting before the risk occurs

Mitigation is the set of actions taken in advance to reduce either the probability or the impact of a risk. In this project mitigation is not a separate phase; it is embedded in the sprint cycle.

1. **Identification.** At every sprint planning session the team asks what could go wrong with the work packages selected for that sprint. Anything raised is written into the risk register with an owner.
2. **Analysis.** Each risk is scored for probability and impact on the 0–5 scale and its exposure computed. This is what produces the risk plan sheet in Section 4.5.
3. **Preventive design.** Risks with exposure of 20 or above generate concrete engineering work packages in the WBS. Data-privacy risk is why encryption and access-control auditing exist as WBS 1.3.1.1; peak-load risk is why load testing exists as WBS 1.2.2.2; integration risk is why the automated API test suite exists as WBS 1.2.2.1. Mitigation is therefore funded and scheduled, not merely intended.
4. **Assignment.** Every risk has exactly one named owner among the four team members, who is responsible for reporting its status.

4.3.2 Risk Monitoring: watching while the project runs

Monitoring is the continuous process of tracking known risks and detecting new ones. The team's monitoring mechanisms are deliberately lightweight so that they are actually performed.

- The risk register is reviewed at the start of every sprint planning and at every retrospective; probability and impact scores are revised in the light of what the sprint revealed.
- Each critical risk carries a *trigger indicator* – an observable signal that the risk is materialising. Rising API error rates in the staging logs indicate the integration risk; response times climbing above two seconds during load tests indicate the peak-load risk; a falling count of daily active test users indicates the adoption risk.
- The daily stand-up is an explicit opportunity to raise a newly noticed risk, and anything raised there enters the register the same day.
- Sprint velocity is tracked against the WBS effort baseline; a persistent shortfall is itself the trigger indicator for the schedule risk.

4.3.3 Risk Management: responding when the risk occurs

Management covers the actions taken once a risk has actually materialised.

1. **Execute the contingency plan** recorded against that risk, rather than improvising under pressure.
2. **Contain the damage** – restore service, revoke access, roll back the release or restore from backup, whichever the plan specifies.
3. **Communicate** to affected users and to the supervising faculty member, with an honest statement of what happened and what is being done.
4. **Review and improve** at the next retrospective, updating the mitigation so that the same failure cannot recur, and re-scoring the risk.

4.4 Worked RMMM Examples

The three risks below are the highest-exposure entries in the register and are worked through the full RMMM procedure to show how it operates in practice.

RMMM Example 1 — R1: Violation of Data Privacy **Exposure 5 × 5 = 25 (Critical)**

Risk description. CampusConnect stores personal information – email addresses, student and employee IDs, phone numbers, private messages, connection lists and profile media – for the entire user base. Unauthorised disclosure of this data would harm real students and would end the university's willingness to endorse the platform.

Mitigation (before it happens).

- Encrypt data in transit with TLS and encrypt sensitive fields at rest; store passwords only as salted hashes. (WBS 1.3.1.1)
- Apply the role-based permission matrix at the API layer, not only in the interface, so that a crafted request cannot read another user's data. (WBS 2.1.1.3)
- Give students explicit privacy controls over which profile fields are visible to whom. (WBS 2.1.2.1)
- Run a scheduled security audit and penetration test before release. (WBS 1.3.1.2)

Monitoring. Automated access-control tests run on every build; the audit log is reviewed weekly for privileged reads; the trigger indicator is any authorisation-failure spike or any read of a record by an account outside the owning role.

Management (if it happens). Revoke the compromised credentials and invalidate all sessions immediately; isolate the affected service; identify the scope of exposure from the audit log; notify affected users and the department; patch the vulnerability and re-run the full audit before restoring the service.

Owner. Security lead (Development Team).

RMMM Example 2 — R3: Server Down Under Peak Load **Exposure 5 × 5 = 25 (Critical)**

Risk description. Campus traffic is not uniform. During section selection, registration and result publication, a large proportion of the student body opens the platform within the same few minutes. A single under-provisioned server would slow to a halt or fail outright at exactly the moment CampusConnect is most needed, which would permanently damage user confidence in it.

Mitigation (before it happens).

- Design the services to be stateless so that horizontal scaling – several small servers behind a load balancer – is possible instead of relying on one large machine. (WBS 1.1.1.1)
- Cache the feed and notice queries and paginate all list endpoints so that a surge does not translate into unbounded database work. (WBS 3.1.1.3)
- Execute load and peak-traffic tests that simulate the registration-day concurrency before release. (WBS 1.2.2.2)
- Schedule high-volume official notices to publish outside the narrow windows when registration traffic peaks.

Monitoring. Server response time, CPU utilisation and error rate are displayed on the platform health dashboard (WBS 5.2.2.1); the trigger indicator is median response time exceeding two seconds or error rate exceeding 1%.

Management (if it happens). Add server instances behind the load balancer; temporarily disable the most expensive non-essential features, such as the ranked feed, and fall back to a simple chronological view; display an honest status message rather than a blank error; after recovery, re-run the load test at

the observed traffic level and raise the baseline capacity.

Owner. DevOps / infrastructure lead (Development Team).

RMMM Example 3 — R8: Module Integration Failure **Exposure 5 × 5 = 25 (Critical)**

Risk description. CampusConnect is assembled from ten interacting modules built in parallel by four people. If the events module and the notification service disagree about a payload, or the moderation module and the feed disagree about what "hidden" means, the platform will not crash — it will quietly show wrong information, such as a notice that never reaches its audience or a removed post that remains visible.

Mitigation (before it happens).

- Define and freeze the interface contract between modules during architecture design, before parallel construction starts. (WBS 1.1.1.1)
- Maintain an automated API and regression test suite that every merge must pass. (WBS 1.2.2.1)
- Integrate continuously through the CI pipeline rather than merging at the end of a sprint. (WBS 1.2.1.1)
- Assign a single technical lead as the arbiter of contract changes, so that no interface changes without both sides being informed.

Monitoring. Build and test results on every commit; the trigger indicator is any failing contract test or a rise in integration-level defects reported during a sprint review.

Management (if it happens). Revert the offending merge to restore a known-good build; reproduce the mismatch with a failing test; correct the contract on both sides and add a permanent regression test; record the incident in the retrospective so the contract review step is strengthened.

Owner. Technical lead (Development Team).

4.5 Risk Plan Sheet

A risk plan sheet is the consolidated list of all risks that threaten the project together with the strategy for mitigating each. Table 4.2 records the ten risks identified for UIU CampusConnect. Probability and impact are scored from 0 to 5 and priority is their product.

Table 4.2: Risk plan sheet for UIU CampusConnect (probability and impact scored 0–5)

Risk ID	Risk Description	Prob.	Impact	Priority	Mitigation Strategy	Owner
R1	**Violation of data privacy.** The system stores personal information such as email, student/employee ID, phone number, messages and friend lists.	5	5	25	End-to-end encryption; enforce access control at the API layer; regular security testing and audit.	Security lead
R2	**Unauthorized access.** A user gains the permissions of another role, for example a student publishing an official notice.	4	5	20	Role-based access control with a documented permission matrix; server-side authorisation validation on every endpoint.	Developer team
R3	**Server down.** During section selection, registration and result publication, many users access the server simultaneously and it fails.	5	5	25	Horizontal scaling behind a load balancer; caching and pagination; load testing before release.	DevOps engineer

continued on next page

Risk ID	Risk Description	Prob.	Impact	Priority	Mitigation Strategy	Owner
R4	**Marketplace fraud.** A user creates a fake listing or provides misleading information about an item.	3	4	12	Seller verification against university identity; reporting mechanism; sold-status controls and listing expiry.	Marketplace manager
R5	**Fake account creation.** Someone from outside the university registers and enters the system.	4	4	16	Restrict registration to the university's official email domain with mandatory verification; manual review of anomalies.	IT admin
R6	**Lost-and-found abuse.** Someone falsely claims that a found item belongs to them.	3	3	9	Require identifying item details before a claim is released; allow users to report false claims; log all claims.	Community manager
R7	**User reluctance.** After the system is built, students show no interest in switching from existing groups.	3	3	9	Promote the platform before launch through clubs and class representatives; user-friendly design validated by sprint reviews with real students.	Product owner
R8	**Module integration failure.** One module does not communicate correctly with another and the system produces incorrect information.	5	5	25	Clearly defined interfaces frozen at design time; automated API and regression testing on every merge; continuous integration.	Technical lead
R9	**Scope creep.** The project has many features and the team keeps adding more, so the work grows beyond the available effort.	4	5	20	Fixed feature baseline recorded in the Vision & Scope chapter; formal approval required for any new requirement; out-of-scope list enforced.	Project manager
R10	**Data loss.** The system stores a large volume of information and server failure could destroy it.	4	4	16	Scheduled automated backups with periodic restore drills; database replication; retention of audit logs.	Database administrator

4.6 Interpreting the Risk Profile

Four risks – R1, R3, R8 and R9 – carry an exposure of 20 or above and are therefore critical. It is worth noting what they have in common: three of them (R1, R3, R8) are consequences of the platform's own ambition. CampusConnect is valuable precisely because it centralises identity, traffic and modules, and centralisation is exactly what makes privacy breaches, load failures and integration defects damaging. The fourth, R9, is a consequence of the same breadth seen from the management side.

This is why the mitigations for these four are not advisory notes but funded elements of the Work Breakdown Structure in Chapter 5: security auditing (1.3.1), load testing (1.2.2.2), automated contract testing (1.2.2.1) and a fixed, documented scope baseline. The remaining six risks are real but either lower in probability or recoverable, and are managed through monitoring and the standard sprint response.

CHAPTER 5

Work Breakdown Structure

5.1 Introduction

A Work Breakdown Structure (WBS) is a deliverable-oriented, hierarchical decomposition of the total scope of work to be carried out by the team in order to achieve the project objectives and produce the required deliverables. It is the bridge between the scope defined in Chapter 2 and the sprint plan of Chapter 3: the scope says *what* the platform is, and the WBS says *what pieces of work* produce it.

For UIU CampusConnect – an integrated campus networking hub combining social feeds, academic section alerts, official notice dissemination, event check-in tools, club administration, group collaboration and peer-to-peer services – the WBS performs three jobs. It fixes the boundary of the system in work terms, so that anything not appearing in the structure is by definition out of scope. It partitions intricate functional areas into small, manageable work packages that one of the four team members can own outright. And it supplies the baseline against which scheduling, effort estimation, sprint sizing, risk monitoring and progress reporting are all performed.

5.2 Principles Applied

The CampusConnect WBS is constructed on the following project management principles.

1. **Total scope capture (the 100% rule).** The structure accounts for 100% of the project scope, including internal, interim and external deliverables. The effort content of every set of child elements sums exactly to that of its parent, with no omissions and no additions. Section 5.7 demonstrates this arithmetically for every branch.

2. **Distinct work boundaries (mutually exclusive elements).** Every work package is uniquely defined with non-overlapping boundaries. This matters concretely in CampusConnect because several modules touch the same data: the notice board, the events module and the messaging module all raise notifications, so the notification engine is placed once, at 2.2.2.1, and consumed by the others rather than being rebuilt three times.

3. **Deliverable-focused structure.** The hierarchy is organised around tangible system outcomes expressed as noun phrases – "QR/token generation and scan-based check-in", not "write the check-in code". This keeps the structure stable even when the implementation approach changes inside a sprint.

4. **Four-level decomposition.** The structure is developed to four numbered levels. Level 1 is the platform, Level 2 the five major system modules, Level 3 the deliverable components inside each module, and Level 4 the terminal work packages that are actually assigned, estimated and tracked. The original three-level structure stopped at component granularity, which was too coarse to schedule against two-week sprints; the fourth level closes that gap.

5. **Optimal granularity (the 40-hour rule).** Level-4 work packages are sized at roughly 8 to 40 direct labour hours – one to five person-days – which is small enough to track meaningfully within a two-week sprint and large enough to avoid micromanagement. With four members over a semester the project baseline is approximately 1,000 person-hours, so one percentage point of effort corresponds to about ten hours; the level-4 packages, sized at 1% to 2%, therefore fall inside the 8–40 hour band by construction.

6. **Balanced work sizing.** Level-3 deliverables carry roughly 3% to 6% of total effort each, which keeps effort evenly distributed across the functional modules and prevents any single component from becoming an unmanageable block. The level-4 packages beneath them are correspondingly sized at 1% to 2%.

5.3 Rules Followed

1. **Cross-functional collaboration.** The breakdown was formulated jointly by all four team members so that inter-module dependencies – feed on identity, notices on notifications, moderation on posts – were respected rather than discovered later.

2. **Systematic identification and numbering.** A standard decimal coding system (1.0, 1.1, 1.1.1, 1.1.1.1) is assigned to every element. These codes are used as the cross-reference key throughout this report: backlog items in Chapter 3 and mitigation actions in Chapter 4 both cite WBS codes, so a mitigation can always be traced to funded work.

3. **Iterative decomposition.** Each level refines the level above it into more granular, well-defined components, stopping as soon as an element is small enough to be estimated confidently and owned by one person.

4. **Single-point accountability.** Every terminal work package is assigned to exactly one owner, accountable for its execution, its quality and its delivery. Shared ownership is not permitted at Level 4.

5.4 Level Structure

Before the decomposition itself, it is worth stating precisely what each of the four levels means in this project, because the word "level" is used loosely in practice. In the CampusConnect WBS a level is defined by what management does with it, not by how deep the numbering happens to go: Level 2 elements are owned, Level 3 elements are estimated and become backlog epics, and Level 4 elements are the units that are actually assigned to a person and tracked to completion. Table 5.1 sets this out.

Table 5.1: Meaning of each WBS level in UIU CampusConnect

Level	Element Type	Example	Management Use
1	Project / platform	UIU CampusConnect	Defines the 100% baseline.
2	System module	2.0 Identity & Social Network	Owned by a module lead; maps to a release theme.
3	Deliverable component	2.1.1 Role-Based Auth & Access Control	Sized at 3–6%; becomes an epic in the backlog.
4	Work package	2.1.1.1 UIU email-domain registration & verification	Sized at 8–40 hours; assigned to one person and tracked in the sprint.

5.5 Level 1 and Level 2 Overview

At the highest level the platform divides into five modules. Their effort allocations sum to 100% of the project.

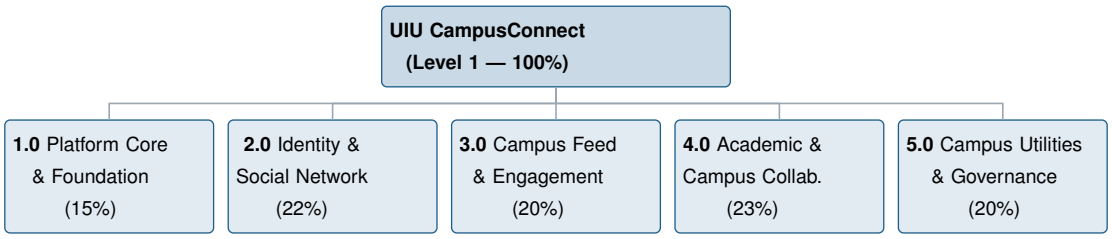


Figure 5.1: Level 1 and Level 2 of the UIU CampusConnect Work Breakdown Structure

Table 5.2: Level-2 modules and their contribution to the project

Code	Module	Effort	Why it carries this share of the work
1.0	Platform Core & Foundation	15%	Architecture, design system, build pipeline, testing infrastructure, security audit and deployment. It produces no user-visible feature, but every other module depends on it and three of the four critical risks are mitigated here.
2.0	Identity & Social Network	22%	Verified registration, roles, profiles, the campus directory, private messaging and the notification engine. Large because it is the trust and communication foundation that objectives O2, O3 and O6 all rest on.
3.0	Campus Feed & Engagement	20%	The feed engine, ranking, global search, post creation, reactions, polls, bookmarks and threaded comments — the daily-use surface of the platform.
4.0	Academic & Campus Collaboration	23%	Official notices, section announcements, events with QR check-in, clubs and academic groups. The largest module because it delivers objectives O1, O3 and O4 and is what distinguishes CampusConnect from a generic social network.
5.0	Campus Utilities & Governance	20%	Marketplace, lost-and-found, reporting, moderation, the admin dashboard and audit logging — objective O5 plus the governance half of O6.
	Total	**100%**	

5.6 Four-Level Decomposition

The five subsections below give the complete four-level structure. Each module is shown as a tree with effort percentages, followed by a dictionary of its Level-4 work packages stating the deliverable that completes each one.

> **Note on corrections to the original allocation.** Two arithmetic inconsistencies in the earlier three-level draft were corrected so that the 100% rule holds at every node. First, the children of 3.2 previously summed to 13% against an 11% parent and have been rebalanced to 5%, 4% and 2%. Second, element 5.1 was shown as 20% although its children summed to 10% and its parent 5.0 totals 20%; it is corrected to 10%, and the two components of 5.2 are allocated 5% each. All Level-2 totals are unchanged.

5.6.1 1.0 Platform Core & Foundation (15%)

Module 1.0 contains the work that produces no user-visible feature and without which no other module can be delivered: the architecture and data model, the design system, the build and test pipeline, and the security audit and deployment that close the project. It is decomposed along the project timeline rather than by feature — design first, then the pipeline that carries every later increment, then the hardening and release work — because that is the order in which the foundation is actually laid. Three of the four critical risks identified in Chapter 4 are mitigated by work packages inside this module.

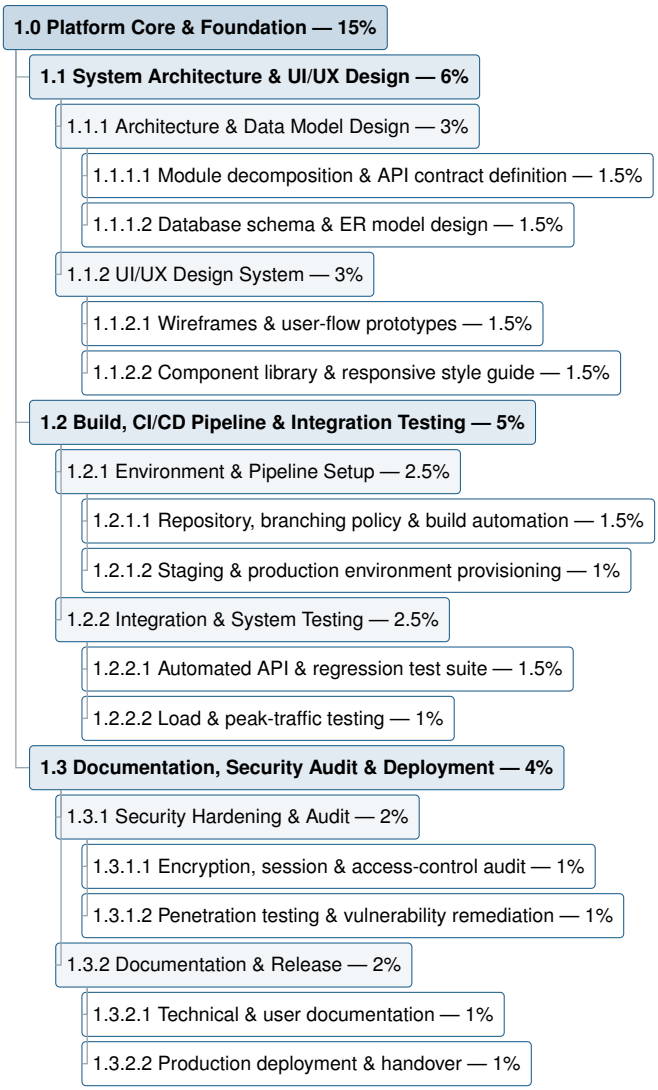


Figure 5.2: Four-level decomposition of Module 1.0 — Platform Core & Foundation

5.6.2 2.0 Identity & Social Network (22%)

Module 2.0 establishes who a user is and how they reach other users. It splits into two halves that are deliberately kept separate: 2.1 covers identity — verified registration, roles, profiles and the campus directory — while 2.2 covers the channels that carry messages and alerts. The split matters because the notification engine at 2.2.2.1 is consumed by the notice board, the events module and the clubs module as well; placing it once, inside a component that owns it, is what stops three teams from each building their own.

5.6.3 3.0 Campus Feed & Engagement (20%)

Module 3.0 is the surface students see every day. It divides into the machinery that decides *what* a user is shown (3.1: aggregation, ranking, search) and the machinery that lets them act on it (3.2: composing posts, reacting, polling, bookmarking and commenting). Keeping discovery separate from interaction means the ranking rules can be changed without touching the post lifecycle, which is useful because ranking is the part most likely to be revised after the first sprint review with real students.

5.6.4 4.0 Academic & Campus Collaboration (23%)

Module 4.0 is the largest in the structure, and it is what distinguishes CampusConnect from a generic social network. It carries the three deliverables that only a university platform can offer: authoritative official

Table 5.3: Level-4 work package dictionary for Module 1.0

Code	Work Package	Completion Deliverable
1.1.1.1	Module decomposition & API contract definition	Signed-off interface specification for all ten modules; the baseline that mitigates risk R8.
1.1.1.2	Database schema & ER model design	Normalised schema covering users, roles, posts, notices, events, clubs, groups and listings.
1.1.2.1	Wireframes & user-flow prototypes	Clickable prototype of the primary flows, validated in a sprint review with student users.
1.1.2.2	Component library & responsive style guide	Reusable UI component set rendering correctly on mobile and desktop.
1.2.1.1	Repository, branching policy & build automation	Version-controlled repository with a pipeline that builds and tests every merge.
1.2.1.2	Staging & production environment provisioning	Two reproducible environments with configuration separated from code.
1.2.2.1	Automated API & regression test suite	Contract tests for every inter-module interface, run on each commit.
1.2.2.2	Load & peak-traffic testing	Test report demonstrating acceptable response times at registration-day concurrency; mitigation for risk R3.
1.3.1.1	Encryption, session & access-control audit	Audit record confirming TLS, hashed credentials and server-side authorisation on every endpoint.
1.3.1.2	Penetration testing & vulnerability remediation	Findings log with all critical and high issues closed; mitigation for risk R1.
1.3.2.1	Technical & user documentation	Deployment guide, API reference and end-user help content.
1.3.2.2	Production deployment & handover	Platform running in production with monitoring enabled and handover completed.

notices and section announcements (4.1), events with enforced capacity and verifiable attendance (4.2), and the club and academic group structures that mirror how campus life is actually organised (4.3). Each of the three is decomposed by workflow — publish, participate, record — because that is how the people using them experience each feature.

5.6.5 5.0 Campus Utilities & Governance (20%)

Module 5.0 pairs two things that appear unrelated but belong together. The peer-to-peer services in 5.1 — marketplace and lost-and-found — are the features most exposed to misuse, because they involve strangers, property and claims of ownership. The governance capability in 5.2 — reporting, moderation, the administrator console and the audit log — is what makes running them responsible. Building the two in the same module ensures that no peer-to-peer feature ships before the mechanism for policing it exists.

5.7 Effort Roll-Up Verification

Table 5.8 demonstrates that the 100% rule holds at every node of the structure: the sum of each element's children equals the element itself, and the five Level-2 modules total 100%.

Table 5.4: Level-4 work package dictionary for Module 2.0

Code	Work Package	Completion Deliverable
2.1.1.1	UIU email-domain registration & verification	Sign-up restricted to university domains with mandatory email confirmation; mitigation for risk R5.
2.1.1.2	Login, session handling & password recovery	Secure authentication with expiring sessions and a verified reset flow.
2.1.1.3	Role assignment & permission matrix	Documented and enforced matrix for student, faculty, official and administrator roles; mitigation for risk R2.
2.1.2.1	Student profile & privacy settings	Editable profile with per-field visibility control.
2.1.2.2	Faculty profile, office hours & research information	Faculty-specific profile supporting objective O3.
2.1.2.3	Profile media upload & validation	Validated image upload with size and type restrictions.
2.1.3.1	Connection / follow request workflow	Request, accept, decline and unfollow operations with notifications.
2.1.3.2	Directory search, block & visibility rules	Campus directory honouring privacy settings and block lists.
2.2.1.1	One-to-one chat service & message store	Persistent private messaging between two verified users.
2.2.1.2	Group conversation threads	Multi-participant threads with membership management.
2.2.1.3	Media attachments & delivery/read receipts	File and image sharing in chat with delivery and read status.
2.2.2.1	Event-driven notification engine	Single shared service consumed by notices, events, clubs and messaging.
2.2.2.2	Push & email delivery channels	Notifications delivered outside the application, supporting objective O1.
2.2.2.3	Notification preferences & digest control	Per-category user preferences and a digest option to prevent alert fatigue.

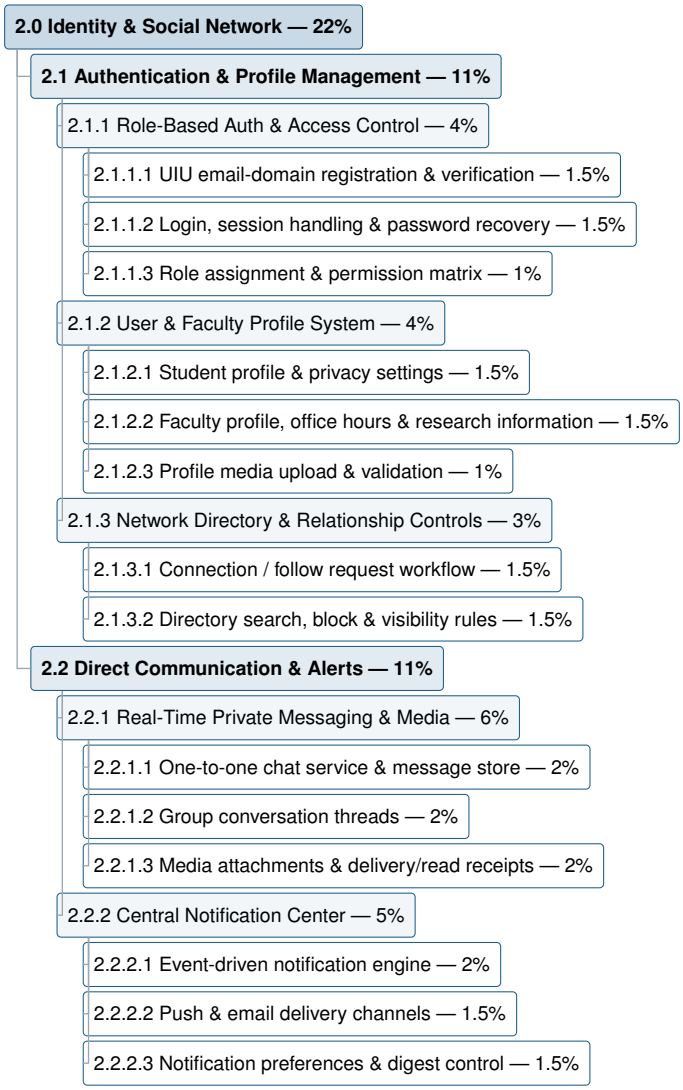


Figure 5.3: Four-level decomposition of Module 2.0 — Identity & Social Network

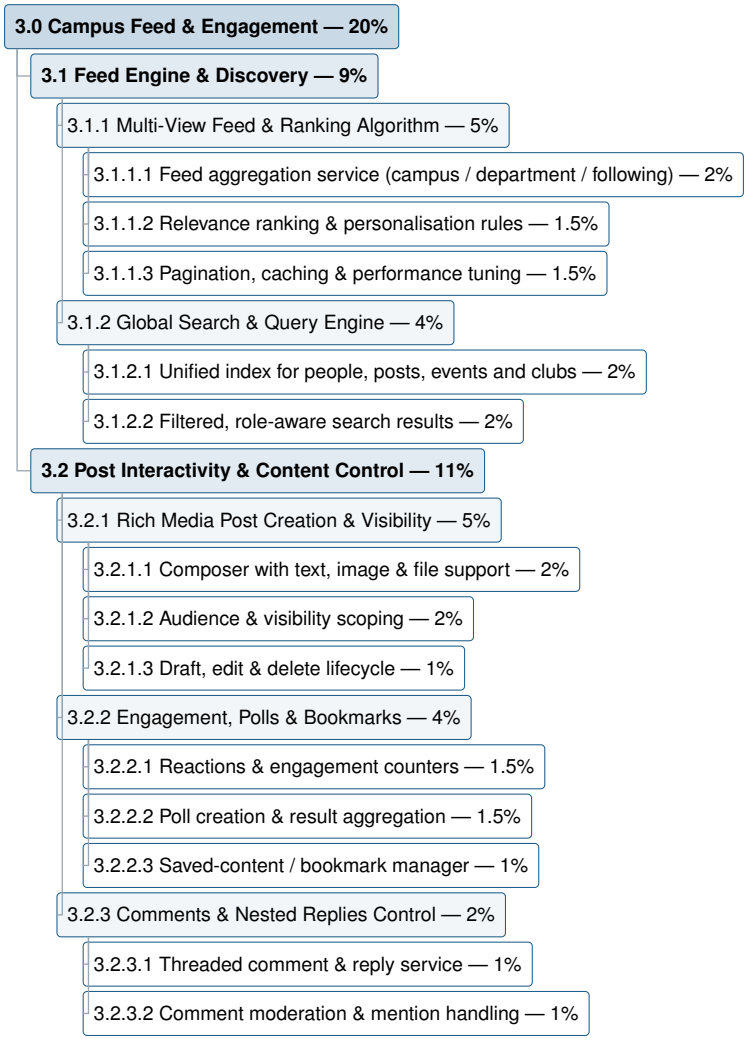


Figure 5.4: Four-level decomposition of Module 3.0 — Campus Feed & Engagement

Table 5.5: Level-4 work package dictionary for Module 3.0

Code	Work Package	Completion Deliverable
3.1.1.1	Feed aggregation service	Feed assembled from campus-wide, departmental and followed sources.
3.1.1.2	Relevance ranking & personalisation rules	Ranking that surfaces a student's own section, batch and clubs first.
3.1.1.3	Pagination, caching & performance tuning	Bounded query cost per request; mitigation for risk R3.
3.1.2.1	Unified search index	Single index spanning people, posts, notices, events and clubs.
3.1.2.2	Filtered, role-aware search results	Results filtered by type and restricted to what the requesting role may see.
3.2.1.1	Composer with text, image & file support	Post composer with validated media upload.
3.2.1.2	Audience & visibility scoping	Author-selected audience: public, department, batch, section or group.
3.2.1.3	Draft, edit & delete lifecycle	Full post lifecycle with edit history retained for moderation.
3.2.2.1	Reactions & engagement counters	Reaction set with accurate aggregate counts.
3.2.2.2	Poll creation & result aggregation	Polls with single or multiple choice and live results.
3.2.2.3	Saved-content / bookmark manager	Personal saved list retrievable across sessions.
3.2.3.1	Threaded comment & reply service	Nested replies with correct ordering and pagination.
3.2.3.2	Comment moderation & mention handling	Mentions raising notifications; comments reportable to the moderation queue.

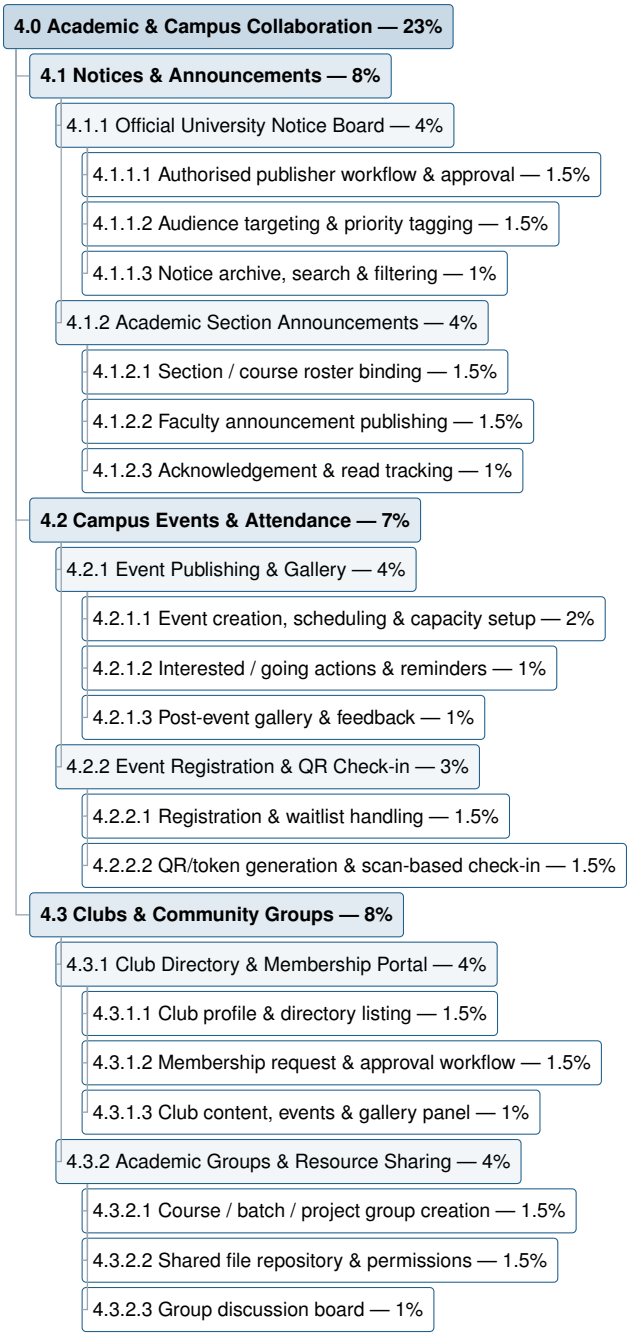


Figure 5.5: Four-level decomposition of Module 4.0 — Academic & Campus Collaboration

Table 5.6: Level-4 work package dictionary for Module 4.0

Code	Work Package	Completion Deliverable
4.1.1.1	Authorised publisher workflow & approval	Only accounts with the official role may publish; approval trail recorded.
4.1.1.2	Audience targeting & priority tagging	Notices targeted to campus, department, batch or section, with priority levels.
4.1.1.3	Notice archive, search & filtering	Full notice history retrievable and filterable, supporting objective O1.
4.1.2.1	Section / course roster binding	Sections linked to enrolled students and the assigned faculty member.
4.1.2.2	Faculty announcement publishing	Section announcements reaching exactly the enrolled students.
4.1.2.3	Acknowledgement & read tracking	Faculty view of who has seen a time-critical announcement.
4.2.1.1	Event creation, scheduling & capacity setup	Event records with venue, schedule and enforced capacity limits.
4.2.1.2	Interested / going actions & reminders	Attendance intent captured and reminders raised through the notification engine.
4.2.1.3	Post-event gallery & feedback	Photo gallery and feedback collection for organiser reporting.
4.2.2.1	Registration & waitlist handling	Registration with automatic waitlist promotion when capacity frees.
4.2.2.2	QR/token generation & scan-based check-in	Unique token per registration and a verified check-in record.
4.3.1.1	Club profile & directory listing	Searchable directory of registered clubs with managed profiles.
4.3.1.2	Membership request & approval workflow	Join requests reviewed by the club committee with roster maintenance.
4.3.1.3	Club content, events & gallery panel	Club-scoped posts, events and media archive surviving committee changes.
4.3.2.1	Course / batch / project group creation	Group types matching the real academic structure.
4.3.2.2	Shared file repository & permissions	Group file area with upload, versioned replacement and access control.
4.3.2.3	Group discussion board	Threaded discussion within each group, supporting objective O3.

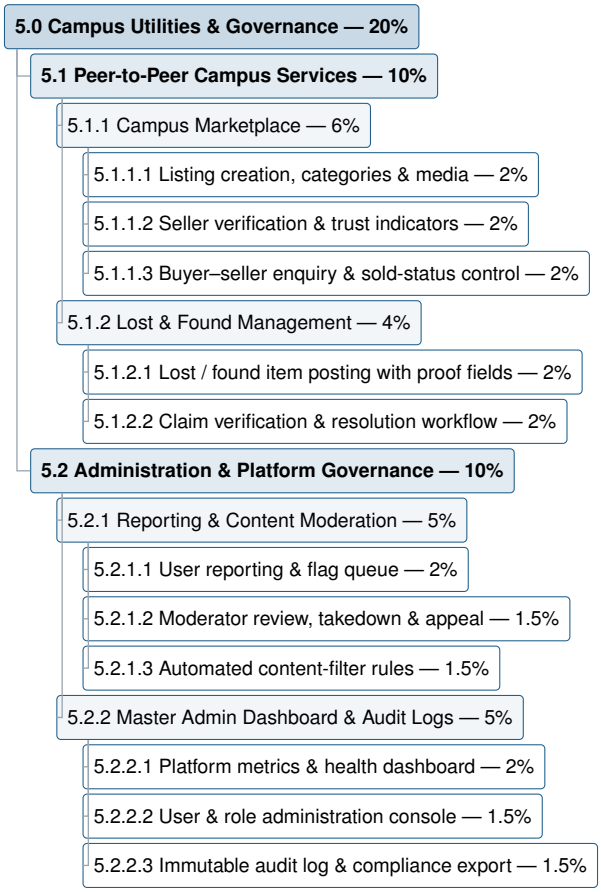


Figure 5.6: Four-level decomposition of Module 5.0 — Campus Utilities & Governance

Table 5.7: Level-4 work package dictionary for Module 5.0

Code	Work Package	Completion Deliverable
5.1.1.1	Listing creation, categories & media	Item listings with category, price, condition and photographs.
5.1.1.2	Seller verification & trust indicators	Every listing attributed to a verified student, with report counts visible; mitigation for risk R4.
5.1.1.3	Buyer–seller enquiry & sold-status control	Enquiry channel and mandatory sold marking with listing expiry.
5.1.2.1	Lost / found item posting with proof fields	Item posts that deliberately withhold one identifying detail for claim verification.
5.1.2.2	Claim verification & resolution workflow	Claimant must supply the withheld detail before release; mitigation for risk R6.
5.2.1.1	User reporting & flag queue	Report action on posts, comments, listings and profiles feeding one queue.
5.2.1.2	Moderator review, takedown & appeal	Defined review path with recorded decisions and an appeal route.
5.2.1.3	Automated content-filter rules	Rule-based pre-screening that flags obvious abuse before a human review.
5.2.2.1	Platform metrics & health dashboard	Live view of active users, response time and error rate; the monitoring surface for risk R3.
5.2.2.2	User & role administration console	Administrator control over accounts, roles and suspensions.
5.2.2.3	Immutable audit log & compliance export	Tamper-evident record of every privileged action; the evidence base for investigating risk R1.

Table 5.8: Verification of effort roll-up across all four WBS levels

Parent Element	Parent %	Children (%)	Sum
1.0 Platform Core	15	1.1 (6) + 1.2 (5) + 1.3 (4)	15 ✓
1.1	6	1.1.1 (3) + 1.1.2 (3)	6 ✓
1.1.1	3	1.5 + 1.5	3 ✓
1.1.2	3	1.5 + 1.5	3 ✓
1.2	5	1.2.1 (2.5) + 1.2.2 (2.5)	5 ✓
1.2.1 / 1.2.2	2.5 each	1.5 + 1 (each)	2.5 ✓
1.3	4	1.3.1 (2) + 1.3.2 (2)	4 ✓
1.3.1 / 1.3.2	2 each	1 + 1 (each)	2 ✓
2.0 Identity & Social	22	2.1 (11) + 2.2 (11)	22 ✓
2.1	11	2.1.1 (4) + 2.1.2 (4) + 2.1.3 (3)	11 ✓
2.1.1 / 2.1.2	4 each	1.5 + 1.5 + 1 (each)	4 ✓
2.1.3	3	1.5 + 1.5	3 ✓
2.2	11	2.2.1 (6) + 2.2.2 (5)	11 ✓
2.2.1	6	2 + 2 + 2	6 ✓
2.2.2	5	2 + 1.5 + 1.5	5 ✓
3.0 Feed & Engagement	20	3.1 (9) + 3.2 (11)	20 ✓
3.1	9	3.1.1 (5) + 3.1.2 (4)	9 ✓
3.1.1	5	2 + 1.5 + 1.5	5 ✓
3.1.2	4	2 + 2	4 ✓
3.2	11	3.2.1 (5) + 3.2.2 (4) + 3.2.3 (2)	11 ✓
3.2.1	5	2 + 2 + 1	5 ✓
3.2.2	4	1.5 + 1.5 + 1	4 ✓
3.2.3	2	1 + 1	2 ✓
4.0 Academic & Collab.	23	4.1 (8) + 4.2 (7) + 4.3 (8)	23 ✓
4.1	8	4.1.1 (4) + 4.1.2 (4)	8 ✓
4.1.1 / 4.1.2	4 each	1.5 + 1.5 + 1 (each)	4 ✓
4.2	7	4.2.1 (4) + 4.2.2 (3)	7 ✓
4.2.1	4	2 + 1 + 1	4 ✓
4.2.2	3	1.5 + 1.5	3 ✓
4.3	8	4.3.1 (4) + 4.3.2 (4)	8 ✓
4.3.1 / 4.3.2	4 each	1.5 + 1.5 + 1 (each)	4 ✓
5.0 Utilities & Governance	20	5.1 (10) + 5.2 (10)	20 ✓
5.1	10	5.1.1 (6) + 5.1.2 (4)	10 ✓
5.1.1	6	2 + 2 + 2	6 ✓
5.1.2	4	2 + 2	4 ✓
5.2	10	5.2.1 (5) + 5.2.2 (5)	10 ✓
5.2.1 / 5.2.2	5 each	2 + 1.5 + 1.5 (each)	5 ✓
Project total (Level 2)		**15 + 22 + 20 + 23 + 20**	**100 ✓**

CHAPTER 6

Wideband Delphi Estimation

6.1 What is Wideband Delphi Estimation?

Wideband Delphi is a consensus-based estimation technique. A small group of experienced people estimate the same work independently, the individual estimates are collected and displayed anonymously, the group then discusses the *assumptions* behind the spread — never the individual numbers — and everyone re-estimates. The cycle repeats until the estimates converge into an acceptable range.

Two properties make the technique valuable. Because the first estimate is made alone, it is not anchored by whoever speaks first or most loudly. And because the discussion is about assumptions rather than numbers, a wide spread becomes useful information: it reveals that two estimators were solving different problems, which is precisely the misunderstanding that would otherwise surface halfway through construction.

6.2 Why Wideband Delphi Suits UIU CampusConnect

The Work Breakdown Structure of Chapter 5 allocates effort in *percentages*, which distribute the work but do not say how large the work is. Wideband Delphi converts that structure into absolute effort in person-days, and does so in a way that suits this project specifically.

- **No historical data exists.** The team has never built a platform of this size, so parametric or analogy-based estimation has nothing to calibrate against. Expert judgement is the only available basis, and Delphi is the disciplined way to combine several experts' judgement.
- **The estimators have genuinely different specialisms.** Front-end, back-end, database and testing perspectives each see a different part of the same task. Estimating separately and then reconciling captures all of them.
- **It exposes hidden assumptions early.** As Section 6.5 shows, the widest disagreement in Round 1 was on the event check-in module, and the cause was an unstated assumption about whether QR handling would be written from scratch. Finding that in an estimation meeting cost an hour; finding it in Sprint 5 would have cost days.
- **It cross-checks the WBS.** The converged per-task estimates can be converted back into percentages and compared with the WBS allocation, which is done in Table 6.7.

6.3 Estimation Session: Three Rounds

Five estimators took part, referred to throughout this chapter as Expert 1 to Expert 5. Their identities are not recorded because Wideband Delphi depends on anonymity: the moderator plots the distribution of estimates without attribution so that no figure carries the authority of the person who wrote it. Four estimators represented the front-end, back-end, database and quality-assurance perspectives on the work; the fifth was external to the project team, which is why several of the largest Round-1 deviations came from that direction.

The estimators worked to a single goal statement — to estimate the total development effort required to build UIU CampusConnect to a deployable first release — and to a single unit, the person-day of eight hours of direct development effort. Estimates measure effort and never calendar time, so waiting time, holidays and examination weeks are excluded. The task list is the twelve Level-2 deliverable components of the Work Breakdown Structure in Chapter 5, so that every estimate maps onto a WBS code.

Three rounds were required before the group agreed that the range was acceptable. The tables below give every estimator's figure for every task in each round, together with the spread.

6.3.1 Round 1 — Independent Estimates

Round 1 is the output of individual preparation: five people estimating the same twelve components alone, with no discussion between them and without seeing each other's figures. The spread is correspondingly wide, and that is the expected and useful result — a first round that already agreed would mean the estimators had influenced one another.

Table 6.1: Round 1 — independent estimates from individual preparation (person-days)

WBS	Task	Expert 1	Expert 2	Expert 3	Expert 4	Expert 5	Min	Max	Mean
1.1	System architecture & UI/UX design	6	10	7	12	9	6	12	8.8
1.2	Build, CI/CD pipeline & integration testing	4	8	5	9	6	4	9	6.4
1.3	Documentation, security audit & deployment	3	7	4	8	5	3	8	5.4
2.1	Authentication & profile management	10	18	12	20	15	10	20	15
2.2	Direct communication & alerts	11	17	20	12	14	11	20	14.8
3.1	Feed engine & discovery	8	14	9	15	12	8	15	11.6
3.2	Post interactivity & content control	10	16	18	13	15	10	18	14.4
4.1	Notices & announcements	7	12	8	14	10	7	14	10.2
4.2	Campus events & attendance (QR check-in)	5	14	7	16	10	5	16	10.4
4.3	Clubs & community groups	8	13	9	12	11	8	13	10.6
5.1	Peer-to-peer campus services	9	15	10	16	13	9	16	12.6
5.2	Administration & platform governance	9	16	11	17	14	9	17	13.4
	Total	**90**	**160**	**120**	**164**	**134**	**90**	**164**	**133.6**

The project totals ranged from 90 to 164 person-days — a spread of 74 days, or 55% of the mean. That is far too wide to plan against, but the pattern of the disagreement was informative: Experts 1 and 3 were consistently low and Experts 2 and 4 consistently high across almost every task, which suggested a systematic difference in what "done" meant rather than random noise. The discussion confirmed it — the low estimators had assumed that testing and documentation were counted once, in task 1.3, while the high estimators had included per-module testing inside every task.

6.3.2 Round 2 — After the First Assumption Review

With the testing boundary settled and the governing assumptions written down and agreed — among them that an existing QR library would be used, that the notification engine is built once and reused, that the marketplace handles no payments, and that section lists can be imported rather than typed in — every estimator re-estimated. The moderator showed only the anonymous distribution, never who had given which number.

The spread narrowed from 74 to 36 person-days. Two tasks remained contentious: 2.2 (direct communication and alerts), where Expert 3 still estimated real-time messaging well above the rest of the group, and 4.2 (campus events and attendance), which is examined in detail in Section 6.5.

6.3.3 Round 3 — Convergence

The third round resolved the two remaining disagreements. Expert 3's messaging estimate was accepted as partly justified once the reasoning was heard — the group raised its own figures for 2.2 rather than Expert 3 lowering theirs, which is exactly the outcome anonymity is meant to make possible — and the QR check-in

Table 6.2: Round 2 — estimates after the first assumption review (person-days)

WBS	Task	Expert 1	Expert 2	Expert 3	Expert 4	Expert 5	Min	Max	Mean
1.1	System architecture & UI/UX design	7	9	8	10	8	7	10	8.4
1.2	Build, CI/CD pipeline & integration testing	5	7	6	7	6	5	7	6.2
1.3	Documentation, security audit & deployment	4	6	5	6	5	4	6	5.2
2.1	Authentication & profile management	12	16	13	17	14	12	17	14.4
2.2	Direct communication & alerts	13	15	17	13	14	13	17	14.4
3.1	Feed engine & discovery	9	12	10	13	11	9	13	11
3.2	Post interactivity & content control	12	15	16	13	14	12	16	14
4.1	Notices & announcements	8	11	9	12	10	8	12	10
4.2	Campus events & attendance (QR check-in)	7	11	8	12	9	7	12	9.4
4.3	Clubs & community groups	9	11	10	11	10	9	11	10.2
5.1	Peer-to-peer campus services	10	14	11	14	12	10	14	12.2
5.2	Administration & platform governance	11	14	12	15	13	11	15	13
	Total	**107**	**141**	**125**	**143**	**126**	**107**	**143**	**128.4**

assumption was settled.

At this point no estimator wished to move further and the group agreed that the remaining range of 17 person-days (13% of the mean) was acceptable for planning. Round 3 is therefore the accepted baseline.

6.4 Convergence Analysis

Table 6.4 summarises how the project total behaved across the three rounds, and Figure 6.1 plots the same data in the form the moderator drew at the start of each round: one mark per estimator on a single axis, with no name attached to any mark.

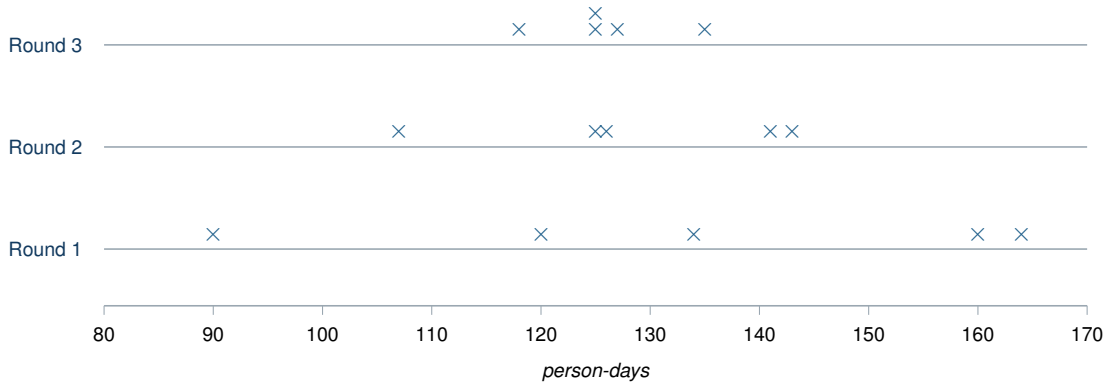


Figure 6.1: Plot of the project totals. Each × is one anonymous estimator; the distribution tightens with every round.

The range fell from 74 person-days to 36 and then to 17, while the mean moved only from 133.6 to 126 person-days. That combination — a collapsing range around a stable centre — is the signature of a healthy Delphi session. A mean that instead drifted steadily in one direction would indicate that the group was being anchored by one estimator rather than reaching agreement.

Table 6.3: Round 3 — converged estimates accepted as the project baseline (person-days)

WBS	Task	Expert 1	Expert 2	Expert 3	Expert 4	Expert 5	Min	Max	Mean
1.1	System architecture & UI/UX design	8	8	7	9	8	7	9	8
1.2	Build, CI/CD pipeline & integration testing	6	6	5	7	6	5	7	6
1.3	Documentation, security audit & deployment	5	5	4	6	5	4	6	5
2.1	Authentication & profile management	14	15	13	15	13	13	15	14
2.2	Direct communication & alerts	14	14	15	13	14	13	15	14
3.1	Feed engine & discovery	11	11	10	12	11	10	12	11
3.2	Post interactivity & content control	13	14	15	14	14	13	15	14
4.1	Notices & announcements	10	10	9	11	10	9	11	10
4.2	Campus events & attendance (QR check-in)	9	9	8	10	9	8	10	9
4.3	Clubs & community groups	10	10	9	11	10	9	11	10
5.1	Peer-to-peer campus services	12	12	11	13	12	11	13	12
5.2	Administration & platform governance	13	13	12	14	13	12	14	13
	Total	**125**	**127**	**118**	**135**	**125**	**118**	**135**	**126**

Table 6.4: Convergence of the project total across three rounds

Round	Minimum	Maximum	Range	Mean	Spread (% of mean)
Round 1	90	164	74	133.6	55.4%
Round 2	107	143	36	128.4	28.0%
Round 3	118	135	17	126	13.5%

6.5 Feature-Level Convergence: 4.2 Campus Events & Attendance

The project total hides the most instructive part of the session. Task 4.2 — event publishing, registration and QR/token check-in — produced the widest disagreement of Round 1 and is worked through here in full, because it shows what the technique is actually for.

Table 6.5: Three-round estimation of WBS 4.2 — Campus Events & Attendance (person-days)

Round	Expert 1	Expert 2	Expert 3	Expert 4	Expert 5	Min	Max	Range	Mean
Round 1	5	14	7	16	10	5	16	11	10.4
Round 2	7	11	8	12	9	7	12	5	9.4
Round 3	9	9	8	10	9	8	10	2	9

6.5.1 What Happened in Each Round

Round 1 — range 11 days (5 to 16). The two extremes were not carelessness; they were two different projects. Expert 1, estimating 5 days, assumed that an open-source QR library would handle generation

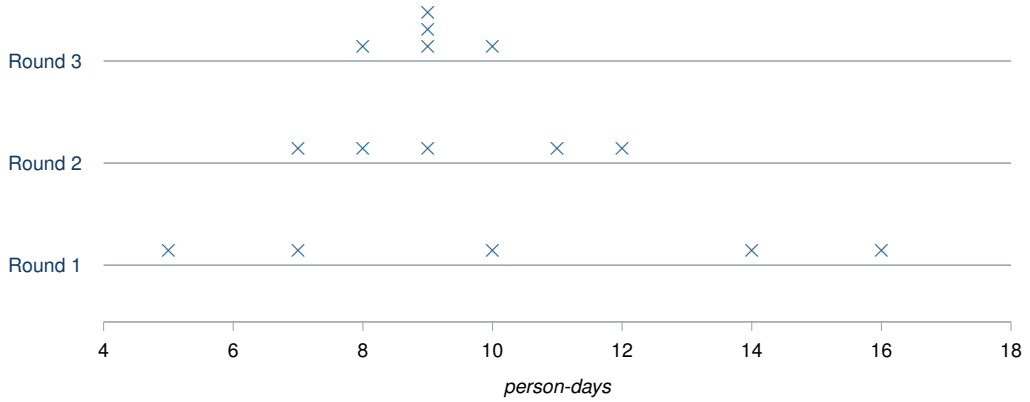


Figure 6.2: Estimation-session convergence for WBS 4.2, Campus Events & Attendance

and scanning, leaving only the registration records and a token field to write. Expert 4, estimating 16 days, assumed the team would implement token generation, signing and scan validation itself, and had also included the post-event gallery and organiser reporting in the same task. Neither assumption had been written down.

Round 2 — range 5 days (7 to 12). The moderator recorded the agreed assumption — *an open-source QR library is used; only the token issue and validation logic is written* — and confirmed against the WBS that 4.2.1.3 (post-event gallery) is a separate work package inside the same component, so it must be counted, but only once. Both extremes moved substantially: Expert 1 rose because the gallery was now inside the boundary, Expert 4 fell because the library was now assumed. The remaining gap was about capacity and waitlist handling, which Experts 2 and 4 considered more intricate than the others did.

Round 3 — range 2 days (8 to 10). The waitlist question was resolved by agreeing that promotion from the waitlist is automatic and requires no administrator screen. With that settled, all five estimates fell within two days of each other and the group accepted a consensus of 9 person-days.

> **Why this matters.** The disagreement was never really about effort; it was about an unwritten assumption. The Delphi session surfaced it in the second week of planning, at the cost of about twenty minutes of discussion. Had it survived until Sprint 5, one member would have started writing a QR token library while another expected the module to be nearly complete — a rework cost of several days, and precisely the kind of silent divergence recorded as risk R8 in Chapter 4.

6.6 Assembling the Tasks

After Round 3 the moderator compiled the master estimate. Following standard practice the sheet records a best case (the sum of the lowest Round-3 estimate for each task), a worst case (the sum of the highest), and the expected value (the sum of the means), so that the plan carries its own uncertainty rather than pretending to a single number.

The consensus estimate is therefore **126 person-days**, or approximately **1008 person-hours**, with a planning band of 114 to 138 person-days. Distributed across four team members this is about 31.5 working days each, which fits a single semester provided the sprint roadmap of Section 3.3.3 is followed and the out-of-scope list of Chapter 2 is enforced.

6.7 Reviewing the Results

The final step is the review meeting, in which the compiled estimate is checked against what the team already believed. Because the WBS allocated effort as percentages and the Delphi session produced absolute days, the two can be compared directly — and disagreement between them would mean that one of the two is wrong.

Table 6.6: Master estimate compiled from Round 3 (person-days)

WBS	Task	Best case	Worst case	Expected	Expected (hours)
1.1	System architecture & UI/UX design	7	9	8	64
1.2	Build, CI/CD pipeline & integration testing	5	7	6	48
1.3	Documentation, security audit & deployment	4	6	5	40
2.1	Authentication & profile management	13	15	14	112
2.2	Direct communication & alerts	13	15	14	112
3.1	Feed engine & discovery	10	12	11	88
3.2	Post interactivity & content control	13	15	14	112
4.1	Notices & announcements	9	11	10	80
4.2	Campus events & attendance (QR check-in)	8	10	9	72
4.3	Clubs & community groups	9	11	10	80
5.1	Peer-to-peer campus services	11	13	12	96
5.2	Administration & platform governance	12	14	13	104
	Project total	**114**	**138**	**126**	**1008**

Table 6.7: Cross-check: Delphi consensus against the WBS effort allocation

WBS	Task	Delphi (days)	Delphi share	WBS share
1.1	System architecture & UI/UX design	8	6.3%	6%
1.2	Build, CI/CD pipeline & integration testing	6	4.8%	5%
1.3	Documentation, security audit & deployment	5	4.0%	4%
2.1	Authentication & profile management	14	11.1%	11%
2.2	Direct communication & alerts	14	11.1%	11%
3.1	Feed engine & discovery	11	8.7%	9%
3.2	Post interactivity & content control	14	11.1%	11%
4.1	Notices & announcements	10	7.9%	8%
4.2	Campus events & attendance (QR check-in)	9	7.1%	7%
4.3	Clubs & community groups	10	7.9%	8%
5.1	Peer-to-peer campus services	12	9.5%	10%
5.2	Administration & platform governance	13	10.3%	10%
	Total	**126**	**100.0%**	**100%**

No component deviates from its WBS allocation by more than 0.5 percentage points. The two estimates were produced by different methods — one by top-down proportional allocation during decomposition, the other by bottom-up expert consensus — so their agreement is meaningful corroboration rather than a tautology. The Delphi result is accepted as the effort baseline for the project, and it feeds directly into three later decisions:

- **Sprint sizing.** With 126 person-days across seven sprints, a sprint carries roughly 18 person-days of work, which is the capacity check applied at every sprint planning session in Chapter 3.
- **The 40-hour rule.** 1008 person-hours spread over 67 Level-4 work packages averages about 15 hours per package, confirming that the decomposition in Chapter 5 reached the intended 8–40 hour granularity.
- **Schedule risk.** The gap between the best case (114 days) and the worst case (138 days) is 24 person-days, roughly 19% of the baseline. That figure is the quantitative basis for the schedule contingency held against risk R9 in Chapter 4.

6.8 Limitations of the Estimate

Two honest qualifications belong with any Delphi result. First, consensus is not correctness: five people can agree and still be wrong together, particularly when four of them share the same limited experience — which is exactly why one estimator was drawn from outside the project team. Second, the estimate is only as good as the assumptions recorded during the session. If the university e-mail verification endpoint turns out to be unavailable, or if payment handling is added to the marketplace, the affected tasks must be re-estimated rather than absorbed silently. The baseline is therefore re-reviewed at every sprint retrospective against the effort actually recorded.

CHAPTER 7

Conclusion

This report has set out the management baseline for UIU CampusConnect. The Vision and Scope chapter fixed what the platform is for and, equally importantly, what it is not: ten functional areas were admitted into the delivery boundary with a stated reason for each, and five capabilities were formally excluded so that the team has written grounds for refusing them later.

The methodology chapter compared the candidate process models against the three defining characteristics of this project – moving requirements, modular but interdependent construction, and a small fixed effort budget – and selected Agile Scrum, mapping its ceremonies onto a seven-sprint semester roadmap. It also stated plainly why each alternative was set aside: Waterfall because the requirements are not knowable in advance and its testing comes too late for our critical risks; Prototype because it is an elicitation technique rather than a process and would accelerate scope creep; Spiral because its risk ceremony is disproportionate to a recoverable student project, though its risk discipline was retained inside the sprint cycle; and unstructured Agile because it answers none of the ownership, cadence and scope-control questions that Scrum answers explicitly.

The risk chapter converted risk from a list of worries into a working procedure. Ten risks were scored for probability and impact, four were found to be critical, and the RMMM discipline was described as the team actually operates it: risks identified at sprint planning, mitigations funded as WBS work packages, trigger indicators watched during the sprint, and a defined contingency executed if a risk materialises. Three of those risks were worked through the full cycle to demonstrate the procedure.

The Work Breakdown Structure decomposed the entire scope into four levels, from the platform down to sixty-seven terminal work packages, each sized for a two-week sprint and each assignable to a single owner. The effort allocations were corrected where the earlier draft did not balance, and the roll-up was verified to 100% at every node.

The Wideband Delphi chapter turned those percentages into absolute effort. Five anonymous estimators, one of them from outside the project team, estimated the twelve Level-2 components independently and then reconciled across three rounds. The spread of the project total fell from 74 person-days in Round 1 to 17 in Round 3 while the mean barely moved, and the session converged on 126 person-days (about 1,008 person-hours). The worked convergence of task 4.2 showed what the technique is really for: an eleven-day disagreement that turned out to be an unwritten assumption about whether QR handling would be built from scratch, caught in a planning meeting instead of in Sprint 5.

Taken together, these five chapters form a single coherent plan. Every objective in Chapter 2 is traceable to work packages in Chapter 5; those packages are priced in Chapter 6 by a method independent of the one that produced the allocation, and the two agree to within half a percentage point; every critical risk in Chapter 4 is mitigated by a funded element of the same structure; and every element is scheduled through the sprint cycle of Chapter 3. That traceability is what makes the plan executable rather than merely descriptive.